

- S02.03 Schema Design patterns
 - How to model data for MongoDB?
 - De/Normalization Reminder
 - Normal forms
 - 1NF
 - Transitive Dependency
 - Transactions and MongoDB
 - ACID Reminder
 - Example: transaction with PostgreSQL
 - Example transaction in mongosh
- Referencing (joins) vs integration (embedding)
 - Integration (embedding)
 - Advantages of integration
 - Limitations of integration
 - Referencing (joins)
 - Advantages of referencing
 - Limitations of referencing
- Schema design and nature of relationships
 - One-to-one
 - One-to-few (tens)
 - One-to-many (hundreds)
 - One-to-squillions
 - 16 MB constraint
 - Example schema for server + log
 - Many-to-many
 - Summary:
 - Schema design patterns
 - Extended reference model
 - The outlier pattern model
 - Other patterns
 - The Bucket Pattern
 - The Computed Pattern
 - The Polymorphic Pattern

- Recap
- 💡 **Conclusion**
- Schema definition and validation
 - Implicit schema during creation
 - 💠 **When should you create a collection manually?**
 - **Example of explicit collection creation**
 - Capped collection
 - 💠 **Limitations of automatic creation**
 - Applying a data type
- Schema Validation in MongoDB
 - How Schema Validation Works
 - Building Blocks of Validation
 - Nested Objects
 - Fine-Tuning Validation Behavior
 - Practice
 - Practice
 - Links

S02.03 Schema Design patterns

MongoDB schema design is the most critical part of database design.

If you design the MongoDB schema like a relational schema, you lose sight of the advantages and power of NoSQL.

Read the article: [MongoDB Schema Design: Best Practices for Data Modeling](#)

How to model data for MongoDB?

It depends. Document databases have a rich vocabulary that is capable of expressing data relationships in a more nuanced way than SQL.

There are many things to consider when choosing a schema.

- **I/O** Is your application read-intensive or write-intensive?
- **Consumption** What data is frequently accessed together?
- **Performance**: What are your performance priorities?
- **Evolution**: How will your data grow and evolve?

In SQL databases, **normalization** consists of distributing data across tables to avoid duplication.

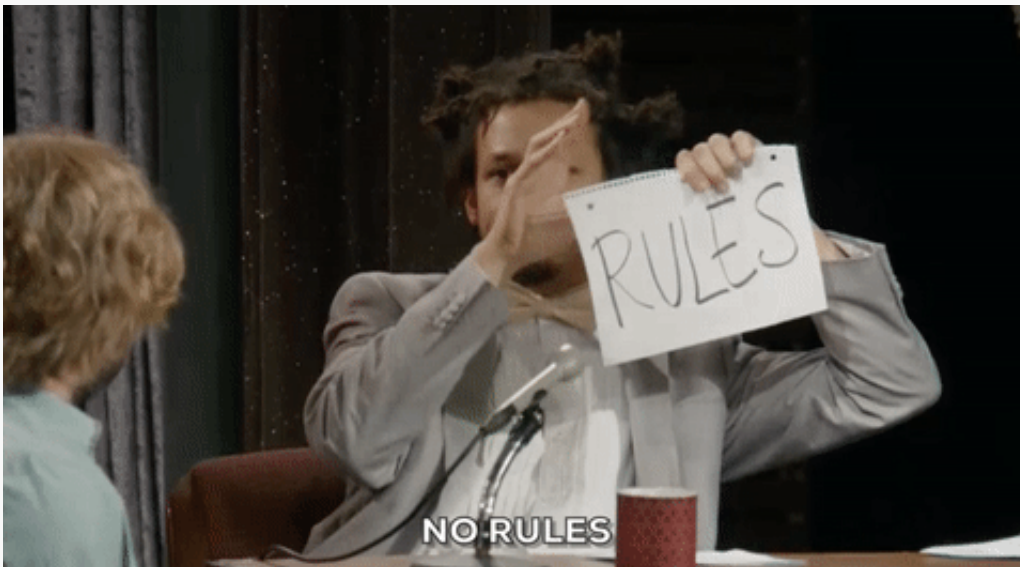
- 1 user
- has several means of transport -> bike, car, ..., private jet
- has a job
- has an address
- has made several visits to a site

In SQL: multiple tables, apply normalization forms (NF1, NF2, ...), etc.

With MongoDB, there are:

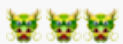
- no formal processes
- no algorithms
- no rules

Which is absolutely not scary !!! 🤔🤔🤔



What matters is designing a schema that works best **for the final application**.

=> Two different applications that use exactly the same data can have very different schemas if the data is used differently.



The application dictates the schema!

De/Normalization Reminder

The general goal of **normalization** is to reduce data **redundancy** and **dependency** by organizing data into **separate, related tables**.

More formally, a database is normalized if:

all column values depend only on the table primary key,

With **Denormalization**, the idea is to have data **redundancy** to simplify queries and make OLAP queries faster.

Redundant data : the same data / info exists in multiple tables

- SELECT queries involve fewer JOINS.
- However INSERT, UPDATE, DELETE queries are more complex as multiple tables must be accounted for. Therefore data integrity is more complex to preserve.

Normal forms

A normal form is a **rule** that defines a level of normalization.

- UNF: Unnormalized form
- 1NF: First normal form
- 2NF: Second normal form
- 3NF: Third normal form

1NF

Each field contains a single value

A relation is in first normal form

if and only if

no attribute domain has relations as elements.

- [Wikipedia: Satisfying 1NF](#)

The table is in 2NF iff :

- The table is in 1NF,
- it has a single attribute unique identifier (UID),
- and every non key attribute is dependent on the entire UID

Some cases of non-compliance with 2NF

- Derived or calculated fields:
 - Relation: `Employee(EmployeeID, Name, BirthDate, Age)`
 - Here, `Age` is derived from `BirthDate`, causing a partial dependency.

A relation R is in 3NF if and only if both of the following conditions hold:

- The relation is in second normal form (2NF).
- No non-prime attribute of R is transitively dependent on the primary key.

Transitive Dependency

A **transitive dependency** occurs when a non-prime attribute (an attribute that is not part of any key)

depends on another non-prime attribute, rather than depending directly on the primary key.

where:

- non-prime attribute: an attribute that is not part of any key

In simple terms: $A \rightarrow B \rightarrow C$ where A is the primary key, but C depends on B instead of directly on A.

For instance in a songs table, we have

- `song_id → artist_name → artist_country`
- `song_id → artist_name → artist_birth_year`

This leads to data redundancy (Artist's info repeated), update anomalies (must change multiple rows to update one artist), and maintenance headaches.

Transactions and MongoDB

Single Document Operations: MongoDB guarantees ACID properties for operations that affect a single document. This is a fundamental aspect of MongoDB's design. So, if all your CRUD operations are contained entirely within **a single document**, then yes, they are ACID compliant!

Embedded Documents (within a single document): When you have embedded documents within a single document, and your CRUD operations only modify data within that **one document**, then the entire operation is ACID compliant because it's still considered a single document operation. For example, updating a field inside an embedded document using `$set` or pushing a new element to an array within the main document using `$push` is ACID compliant.

Multi-document operations (without transactions): Not ACID compliant. You get atomicity at the document level, but not across multiple documents.

ACID Reminder

1. **Atomicity:** A transaction is either fully executed or canceled in case of failure (all or nothing).
2. **Consistency:** A transaction brings the database from one valid state to another valid state, respecting integrity constraints.
3. **Isolation:** Concurrent transactions execute without interfering with each other, as if they were executed sequentially.
4. **Durability:** Once validated, a transaction is permanently recorded, even in case of failure.

Example: transaction with PostgreSQL

```
-- Start a transaction
BEGIN;

-- Perform multiple operations within the transaction
-- Assuming you have tables 'accounts' and 'transactions'

-- Example 1: Update the balance of account 1 (Debit)
UPDATE accounts
SET balance = balance - 100
WHERE account_id = 1;

-- Example 2: Update the balance of account 2 (Credit)
UPDATE accounts
SET balance = balance + 100
WHERE account_id = 2;

-- Example 3: Insert a record into the 'transactions' table to log the transaction
INSERT INTO transactions (account_id_from, account_id_to, amount, transaction_date)
VALUES (1, 2, 100, NOW());

-- Commit the transaction
COMMIT;
```

Example transaction in mongosh

```
// Connect to MongoDB
conn = new Mongo();
db = conn.getDB("mydatabase");

// Start a session
let session = db.getMongo().startSession();

// Start a transaction within the session
session.startTransaction();

try {
  // Perform multiple operations within the transaction

  // Insert a document into collection1
  let result1 = db.collection1.insertOne(
    { name: "Bob", age: 40 },
    { session: session }
  );

  // Update a document in collection2
  let result2 = db.collection2.updateOne(
    { _id: 1 },
    { $inc: { value: 20 } },
    { session: session }
  );

  // Print the results (including $txnNumber and lsid, if available)
  print("Insert Result 1:", result1);
  print("Update Result 2:", result2);

  // Commit the transaction
  session.commitTransaction();
  print("Transaction committed successfully.");
} catch (error) {
  // Abort the transaction in case of an error
  session.abortTransaction();
  print("Transaction aborted. Error:", error);
} finally {
  // End the session
  session.endSession();
}
```

[Back to data schema types](#)

Referencing (joins) vs integration (embedding)

- Integration <-> Denormalization

- Referencing <-> Normalization

Let's compare the following 2 approaches:

- Referencing: the document has a reference to another document

```
JavaScript
// user collection
{
  "_id": "user_id",
  "name": "Bart",
  "school": "Springfield elementary"
}

// transportation collection
{
  "_id": "some_id",
  "user_id": "user_id",
  "type": "bike",
  "avg_speed": "30kph",
},
{
  "_id": "some_id",
  "user_id": "user_id",
  "type": "Skate board",
  "avg_speed": "10kph",
}
```

- integration / nesting (embedding): the sub-document is part of the main document

```
JavaScript
// user collection
{
  "_id": "user_id",
  "name": "Bart",
  "school": "Springfield elementary",
  "transportation": [
    {
      "type": "bike",
      "avg_speed": "30kph",
    },
    {
      "type": "Skate board",
      "avg_speed": "10kph",
    }
  ]
}
```

Integration (embedding)

Advantages of integration

- We retrieve all information in a single query.
- we avoid joins
- a single operation to update embedded data.

For transactions:

- By default, all CRUD operations on a single document are ACID compliant.

Limitations of integration

- Document size negatively impacts query performance. We must be careful not to put everything in a document, but restrict to relevant information.
- MongoDB documents are limited to a size of **16 MB**.

There is therefore a balance to be found between information completeness and document size

Referencing (joins)

The other option for schema design is to reference another document using the unique object ID of a document and connect them using the `$lookup` operator.

Referencing works the same way as the `JOIN` operator in an SQL query. It allows data to be distributed to perform more efficient and scalable queries, while maintaining relationships between data.

Advantages of referencing

- By distributing data, we get smaller documents and therefore less likely to reach the 16 MB per document limit.
- Some data is not accessed as frequently as other data. So it makes sense to separate them from the main data.
- Referencing reduces data duplication.

Limitations of referencing

- MongoDB is less efficient for joins than an SQL database.

Schema design and nature of relationships

We must consider the nature of relationships between entities

One-to-one

Modeled as key-value pairs in the database.

For example:

- train <-> number of passengers
- user <-> creation date
- tree <-> variety
- human <-> height

One-to-few (tens)

A small sequence of data associated with the main entity.

- a person has a few addresses (1, 2, ... 5)
- a playlist has a few songs
- a recipe has a few ingredients
- a bus line has multiple stops

One-to-many (hundreds)

- a gym has many users
- a product has many parts
- an instagram account can have many followers

One-to-squillions

Potentially millions of embedded documents

- university <-> thousands of students
- celebrity instagram account <-> millions of followers, comments
- server <-> log events
- IoT: millions of events

16 MB constraint

A MongoDB record cannot exceed 16 MB. In the case of One to Millions, this might become problematic.

Example schema for server + log

Consider a server logging application where each server records a significant amount of events.

So we have 2 entities: `server` and `event`.

3 options

- the `server` document integrates all `events` associated with the `server`. high probability that this will exceed 16 MB per document quite quickly
- separate the 2 collections and do a `$lookup`. But it's slower to retrieve all events from a server
- reverse the integration and insert the `server` name into each `event` document! data duplication but query speed and no risk of exceeding the 16MB size

Many-to-many

Example of a project planning application:

- a user can have several tasks
- a task can have several assigned users.

An efficient schema is to store

- user IDs in the task document
- task IDs in the user document.

Users:

```
{
  "_id": ObjectId("AAF1"),
  "name": "Kate Pineapple",
  "tasks": [ObjectId("ADF9"), ObjectId("AE02"), ObjectId("AE73")]
}
```

JavaScript

Tasks:

```
{
  "_id": ObjectId("ADF9"),
  "description": "Write a blog post about MongoDB schema design",
  "due_date": ISODate("2014-04-01"),
  "owners": [ObjectId("AAF1"), ObjectId("BB3G")]
}
```

JavaScript

In this example, you can see that each `user` has a sub-array of related `tasks`, and each task has a sub-array of owners for each item in our task application.

Summary:

- **One-to-one** - Prefer key-value pairs in the document
- **One-to-few** - Prefer embedding
- **One-to-many** - Prefer embedding

- **One-to-squillions** - Prefer referencing
- **Many-to-many** - Prefer cross-referencing

General rules for MongoDB schema design:

- **Rule 1:** favor embedding, unless there is a good reason not to.
- **Rule 2:** the need to access an object independently is a good reason not to embed it.
- **Rule 3:** avoid joins and lookups if possible, but don't be afraid if they can provide a better schema design.
- **Rule 4:** arrays should not grow without limit. When there are more than a few hundred embedded documents, it's better to reference them; Similarly, when there are more than a few thousand embedded documents, don't use an array of references by ID. Finally, high cardinality arrays (many potential values) are a good reason not to embed.
- **Rule 5:** as always, with MongoDB, how you model your data depends entirely on your application's data access patterns. You want to structure your data to match how your application queries and updates it.

Schema design patterns

Let's examine 2 Schema models to illustrate how the application dictates data schema design.

Extended reference model

Let's review this article

<https://www.mongodb.com/developer/products/mongodb/schema-design-anti-pattern-massive-arrays/>

One of the rules of thumb when modeling data in MongoDB is to say that data that is accessed at the same time should be stored together.

-> A building has many employees: potentially too many for the 16 MB document limit.

We reverse the situation with

-> The employee belongs to a building: we integrate the building information into the employee document.

If the application frequently displays information about an employee and their building together, this model is probably wise.

Problem: we have way too much data duplication.

Updating a building's information involves updating all employee documents.

So, let's separate employees and building into 2 distinct collections and use `$lookups`.

But `$lookups` are expensive.

We therefore use the [extended reference pattern](#) where we duplicate some, but not all, of the data in both collections. We only duplicate data that is frequently accessed together.

For example, if the application has a user profile page that displays information about the user as well as the name of the building and region where they work in, we integrate the building name and region into the employee document but the other building-related info in a building specific collection.

The outlier pattern model

The outlier pattern: only a few documents have a huge amount of embedded documents.

<https://www.mongodb.com/blog/post/building-with-patterns-the-outlier-pattern>

Consider a collection of books and the list of users who bought the book.

```
JavaScript
{
  "_id": ObjectId("507f1f77bcf86cd799439011")
  "title": "A boring story",
  "author": "Sam K. Boring",
  ...,
  "customers_purchased": ["user00", "user01", "user02"]
}
```

Most books only sell a few copies. This is the long tail of book sales.

For most books we can simply embed the list of buyers (ID and some relevant info) in the book document.

A small number of books sell millions of copies. Impossible to nest buyers in the book doc.

By adding a field, a flag, or indicator, that signals that the book is very popular, we can adapt the schema according to this indicator.

```
JavaScript
{
  "_id": ObjectId("507f191e810c19729de860ea"),
  "title": "Harry Potter",
  "author": "J.K. Rowling",
  ...,
  // we avoid integrating buyers for this book
  // "customers_purchased": ["user00", "user01", "user02", ..., "user9999"],
  "outlier": "true"
}
```

In the application code, we test for the presence of this indicator and handle the data differently if the indicator is present. For example by referencing buyers of very popular books instead of embedding them.

The outlier pattern is frequently used in situations where popularity is a factor, such as in social media relationships, book sales, movie reviews,

Other patterns

The Bucket Pattern

 When to use it?

- When dealing with time-series data (logs, IoT, events).
- To reduce the number of documents.

 Example: Store sensor readings in buckets

Instead of creating a new document for every sensor reading, group them into time buckets.

```
{
  "_id": "sensor_1_2024-02-05",
  "sensor_id": 1,
  "date": "2024-02-05",
  "readings": [
    { "time": "08:00", "value": 22.5 },
    { "time": "08:30", "value": 23.1 }
  ]
}
```

JavaScript

-  Advantages: Reduces document count, improves query performance.
-  Limitations: Harder to update individual readings.

The Computed Pattern

 When to use it?

- When you frequently compute a value that rarely changes.
- To avoid re-calculating values on every query.

 Example: Store total order value instead of computing every time

```
{
  "_id": 101,
  "user_id": 1,
  "items": [
    { "name": "Laptop", "price": 1200 },
    { "name": "Mouse", "price": 50 }
  ],
  "total_price": 1250
}
```

- 💡 Advantages: Improves read performance.
- ⚠️ Limitations: Must ensure updates stay synchronized.

The Polymorphic Pattern

📌 When to use it?

- When documents in the same collection have different attributes.
- Useful when storing similar but distinct entities.

✅ Example: Store users and admins in the same collection

```
{
  "_id": 1,
  "type": "user",
  "name": "Alice",
  "email": "alice@example.com"
}

{
  "_id": 2,
  "type": "admin",
  "name": "Bob",
  "email": "bob@example.com",
  "admin_permissions": ["manage_users", "delete_posts"]
}
```

- 💡 Advantages: Avoids multiple collections.
- ⚠️ Limitations: Queries may need filtering by type.

Recap

Pattern	Use Case	Example
Embedding	Small, frequently accessed data	Store a user with their addresses
Referencing	Large, reusable data needing updates	Users & orders stored separately
Bucket	Time-series or grouped data	Sensor readings stored in time buckets
Outlier	Avoids large documents with exceptions	Move excess comments to a separate collection
Computed	Avoids expensive calculations	Store total order price in the document
Polymorphic	Multiple object types in one collection	Users & admins stored together



Conclusion

- **Embedding** is best for **fast reads** but can lead to large documents.
- **Referencing** is better for **scalability** but requires joins (`$lookup`).
- **Bucket** pattern is great for **time-series data**.
- **Outlier** prevents performance issues due to **large documents**.
- **Computed** speeds up queries by **precomputing values**.
- **Polymorphic** allows flexibility **within the same collection**.

Choosing the **right pattern** depends on **query patterns, data size, and update frequency**. 🚀

Schema definition and validation

Without explicit validation it is possible to put anything in a collection.

For example, the user's age

- 32 (int)
- "32" (string)
- thirty-two
- 432
- null
- ""
- "orange ducklings on skateboards"

etc etc.

The absence of validation regarding acceptable values in a collection transfers complexity to the

application level.

It's chaos!

Implicit schema during creation

In MongoDB, **collections are created automatically** when a **first document is inserted** into a non-existent collection. It is not necessary to create it manually beforehand.

Example

```
db.characters.insertOne({ name: "Alice", age: 25 });
```

JavaScript

✅ If the `characters` collection doesn't exist, MongoDB creates it automatically and inserts the document.

When should you create a collection manually?

You might want to use `createCollection()` if you need to: 1. **Define specific parameters** like a **capped collection** (fixed size). 2. **Apply validation rules** to ensure data integrity.

Example of explicit collection creation

```
db.createCollection("users", {  
  capped: true,  
  size: 1024  
});
```

JavaScript

✅ This creates a **capped collection** with a **fixed size of 1024 bytes**.

Capped collection

A capped collection maintains insertion order and automatically **overwrites** the oldest documents when the size limit is reached. Think of it like a circular buffer or ring buffer: once it's full, new data pushes out the oldest data.

Very fast insertions (no need to allocate new space)

Great for server logs, message queues, real-time data streams for instance.

Limitations of automatic creation

- The collection is **created only when a document is inserted**, not when executing an empty `find()`.
- The collection is created **without validation rules** (unless explicitly defined).
- Indexes **are not created automatically** (except for `_id`).

 **Conclusion:** Yes, MongoDB **automatically creates collections** when inserting data, but explicit creation is useful for advanced parameters.

Applying a data type

Declare the data type when defining the schema

```
db.createCollection("movies", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["year", "title"], // Fields that must be present
      properties: {
        year: {
          bsonType: "int", // Forces `year` to be an integer
          required: "true",
          description: "Must be an integer and is required"
        },
        title: {
          bsonType: "string",
          description: "Movie title, must be a string if present"
        },
        imdb: {
          bsonType: "object", // Nested object for IMDb data
          properties: {
            rating: {
              bsonType: "double", // IMDb rating must be a float
              description: "Must be a float if present"
            }
          }
        }
      }
    }
  }
})
```

Key points:

- **bsonType**: specifies the BSON data type for the field (e.g., int, string, array, object).
- **required**: ensures that specific fields are mandatory.
- **properties**: defines constraints for each field.
- **description**: adds a useful description for validation errors.

MongoDB supports schema validation starting from version 3.6, which allows you to enforce data types and other constraints on fields within a collection.

This is achieved using the `$jsonSchema` operator when creating or updating a collection.

<https://www.digitalocean.com/community/tutorials/how-to-use-schema-validation-in-mongodb>

When you add validation rules to an existing collection, the new rules will not affect existing documents until you try to modify them.

Schema Validation in MongoDB

MongoDB is known for its flexibility - you can store documents without predefined structures. However, as your application grows, you might want to ensure your data follows certain rules. This is where schema validation comes in.

How Schema Validation Works

When you create a collection with validation, MongoDB will check every new document (and updates to existing ones) against your rules. Here's what the basic structure looks like:

```
db.createCollection("collectionName", {  
  validator: {  
    $jsonSchema: {  
      bsonType: "object",  
      required: ["field1", "field2"],  
      properties: {  
        field1: { type: "string" },  
        field2: { type: "number" }  
      }  
    }  
  }  
})
```

JavaScript

The `$jsonSchema` keyword tells MongoDB that we're using JSON Schema validation. Inside this schema, we define our rules using various building blocks.

Building Blocks of Validation

The most fundamental components are:

First, we specify which fields are mandatory using `required`. These fields must be present in every document.

Next, we define `properties` - this is where we describe what each field should look like. For each property, we can specify its type and additional rules. For example, if you're storing someone's age, you might want to ensure it's always a number and perhaps even set a minimum / maximum value.

Let's look at how we handle more complex structures:

Nested Objects

Sometimes your data has natural hierarchies. For instance, an address isn't just one piece of information - it has streets, cities, and zip codes. Here's how you validate nested structures:

```
properties: {
  address: {
    bsonType: "object",
    required: ["city"], // City is mandatory in addresses
    properties: {
      city: { type: "string" },
      zip: { type: "string" }
    }
  }
}
```

JavaScript

Fine-Tuning Validation Behavior

MongoDB gives you control over how strict your validation should be. You can set two important behaviors:

The `validationAction` determines what happens when a document fails validation: - "error" (default): Reject the document completely - "warn": Accept the document but log a warning (great during development!)

The `validationLevel` controls when validation happens: - "strict" (default): Check all inserts and updates - "moderate": Skip validation for existing documents that don't match the schema

Remember that validation only happens when documents are modified or inserted. Existing documents won't be validated until you try to update them. This makes it safe to add validation to collections that already contain data.

Through schema validation, MongoDB offers a balance between flexibility and control. You can start with a loose schema during early development, then gradually add more validation rules as your application's needs become clearer. This progressive approach to data quality helps ensure your database remains both reliable and adaptable.

Practice

Small example on tree datasets

```
{
  "idbase":249403,
  "location_type":"Tree",
  "domain":"Alignment",
  "arrondissement":"PARIS 20E ARRT",
  "suppl_address":"54",
  "number":null,
  "address":"AVENUE GAMBETTA",
  "id_location":"1402008",
  "name":"Linden",
  "genre":"Tilia",
  "species":"tomentosa",
  "variety":null,
  "circumference":85,
  "height":10,
  "stage":"Adult",
  "remarkable":"NO",
  "geo_point_2d":"48.86685102642415, 2.400262189227641"
},
```

write a schema and validator for this data

the validator must apply

- height ≥ 0 and < 100
- stage 'Adult', 'Young', 'Mature'

Practice

take a JSON version of the trees dataset where columns with null values have been removed

1. without constraints
 - insert all data without validation
 - query to check absurd values: height, geolocation
2. more controlled approach
 - write a schema and validator: using MongoDB's \$jsonSchema validator to "dry run" your data validation.
 - check the number of documents ignored
 - write a validator that recovers as many documents as possible while excluding absurd values
3. index
 - add a unique index on geolocation

in MongoDB, the schema validator is usually created with the collection, not as a separate step

Links

Schema design patterns: <https://learn.mongodb.com/courses/schema-design-patterns>